CSM GLEAMING Meta

COMPUTER SCIENCE MENTORS 61A

November 2, 2026 – November 6, 2026

1 Gleam basics

Gleam is a functional programming language, a paradigm where programs are built by applying and composing functions, and function definitions are trees of expressions mapping values to other values rather than sequences of statements that update program state.

Gleam is also statically-typed and type-safe. Type safety is the extent to which a language prevents type errors, such as adding a string to an integer. Type enforcement can be static (at compile time), dynamic (at run-time), or both. In static type systems, type safety usually guarantees that an expression's eventual value belongs to its static type.

When writing functions that operate on a given input, Gleam requires that you handle any theoretically possible input case, even if you've potentially handled such a case earlier. For instance, the function `list.max` (which behaves very similarly to Python's `max` function) does not return an integer directly, but a `Result`. This covers an empty list being passed into `max`: it returns `Ok(value)` or `Error(Nil)` even if you've already ruled out the empty case, guaranteeing an `Ok` result.

Similarly, a `case` statement in Gleam must handle every theoretically possible case, even if that case may not be possible in the context of your own code. For instance, say you have a function that exclusively outputs even integers, and you use one of those integers `n` in a `case` statement:

```
case n % 2 {
    0 -> //some code for the case in which n is even
    _ -> //some code for the case in which n is odd
}
```

Gleam will require you to include the catch-all _ case, regardless of whether `n` could be odd. Even writing a `1` case is not enough: Gleam cannot prove that `n % 2` is only ever `0` or `1` (it is `-1` for negative odd `n`), so an `Int` case always needs _.

Created by Ana Abell, Terry Hsufeedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1. Implement `factorial`!

```
pub fn factorial(n: Int) -> Int {
  case n > 0 {
    True -> _____________ * ___________________________
    False -> _____________
  }
}
```

```
pub fn factorial(n: Int) -> Int {
  case n > 0 {
    True -> n * factorial(n - 1)
    False -> 1
  }
}
```

This is an implementation of the standard factorial in Gleam. Instead of if statements, Gleam uses case matching. `n > 0` evaluates to a `Bool`, which is a sum type between `True` and `False`. `case` takes this `Bool` type and evaluates to a different value for each value it can take: `True` or `False`.

In regular recursion, our very first function will open a new frame, and each new frame will open new frames until we reach our base case. Typically, every frame must wait for subsequent frames to finish evaluating and return before they can return. That means that right before the base case, we have a lot of calls awaiting the call underneath it before doing something. Because each frame takes up physical space on our computer, recursion can be space inefficient. One way of solving this is tail recursion. If we know that we can write our recursive function in such a way that a previous frame doesn't need to do anything with the later frames, we can close previous frames once we're done with them- saving space on our computer. Here's an example:

```
pub fn factorial(x: Int) -> Int {
  factorial_helper(x, 1)
}

fn factorial_helper(x: Int, curr: Int) -> Int {
  case x {
    0 | 1 -> curr
    _ -> factorial_helper(x - 1, curr * x) // The last thing this function
       does is call itself
  }
  //credit to the Gleam language website for this example
}
```

We've defined this helper function such that it checks our base condition and then immediately returns a recursive call. Because it doesn't do anything else with the result of the recursive call, we have no reason to keep the original frame open, and it will close the first frame to save computer memory. Ultimately, the very last recursive call will be returned with our correct value. Tail recursion only works when a language is tail-call optimized, meaning that the language knows to close unnecessary recursive frames. Python isn't tail-call optimized, so even if you write a tail recursive function, it won't be any more memory efficient. However, Gleam is tail-call optimized, so we can take advantage of this!

2. Implement `sum`, a tail recursive function that returns the sum of all elements of a list of integers.

```
fn sum_helper(l: List(Int), curr: Int) -> Int {
  case l {
    [] -> _______
    [first] -> _______ + _______
    [first, ..rest] -> ___________________________________
  }
}

pub fn sum(l: List(Int)) -> Int {
  sum_helper(_______, _______)
}
```

```
pub fn sum_helper(l: List(Int), curr: Int) -> Int {
  case l {
    [] -> curr
    [first] -> curr + first
    [first, ..rest] -> sum_helper(rest, curr + first)
  }
}

pub fn sum(l: List(Int)) -> Int {
  sum_helper(l, 0)
}
```

The purpose of this question is to demonstrate how we can define a helper function to make our function tail recursive when our original function does not have enough arguments. By introducing the function `sum_helper` with the extra argument `curr`, we are able to store our intermediate results in `curr` so that the final call simply returns `curr`. Importantly, we have to define the helper function *outside* of **`sum`**. Gleam does not support nested named functions, so every nested function has to be an anonymous lambda using a `let`, but let-bound lambda functions can't refer to themselves, so they can't be used in recursion.

This question also introduces way to case match lists in Gleam. `[]` matches an empty list, `[first]` matches a list with one element called `first`, and `[first, ..rest]` matches a list with at least one element: `first` for the first element and `rest` for the rest of the list. Cases are checked in order, so here `[first, ..rest]` only sees lists with two or more elements, since `[first]` comes before it.

Gleam supports generics, which let you write functions that work with any type. Instead of naming a concrete type, you use a type variable, written with a lowercase name, and it stands in for whatever type the function is called with. Here are some examples:

```
pub fn main() {
  let add_one = fn(x) { x + 1 }
  let exclaim = fn(x) { x <> "!" }
  echo twice(10, add_one) // Outputs 12. The type variable is Int.
  echo twice("Hello", exclaim) // Outputs "Hello!!". The type variable is
      String.
}

// The name `value` refers to the same type multiple times
fn twice(argument: value, my_function: fn(value) -> value) -> value {
  my_function(my_function(argument))
}
//credit to the Gleam language website for this example
```

We also don't have to restrict ourselves to only doing this with functions! We can do this in type definitions as well (see the `Tree` type in Gleam).

3. Implement `reduce`, a tail recursive function that applies a two-argument function `f` on the elements of a list from left to right, starting with `start`.

```
pub fn reduce(l: List(a), f: fn(a, a) -> a, start: a) -> a {
  case l {
    [] -> _______
    [first] -> ___________________________________
    [first, ..rest] -> ___________________________________
  }
}
```

```
pub fn reduce(l: List(a), f: fn(a, a) -> a, start: a) -> a {
  case l {
    [] -> start
    [first] -> f(start, first)
    [first, ..rest] -> reduce(rest, f, f(start, first))
  }
}
```

The purpose of this question is to demonstrate the use of generics in Gleam. **`reduce`** accepts a list of any type, and only enforces that the type of the elements of the list is the same as the type of `f` (both the arguments and return value), `start`, and the return type of **`reduce`**. **`reduce`** is tail recursive because the only case where a recursive call is made makes no modification to the return value of the recursive call.

2 Algebraic Data Types

An algebraic data type is a type built from other types in two ways: by combining them, called a product type, or by choosing between them, called a sum type. We can implement a product type using a tuple. `#(Int, Bool, String)` contains an `Int`, a `Bool`, and a `String`. Gleam allows defining sum types by specifying variants of a type. Each variant has its own constructor. We instantiate sum types by calling a constructor of a variant.
A `LinkedList(a)` is either a `Link` holding a value and another list, or `Empty`. Each variant is itself a product of its fields. To use a sum type, you pattern match with `case`.

```
pub type LinkedList(a) {
  Link(first: a, rest: LinkedList(a))
  Empty
}
```

1. Implement `append`, which takes in two `LinkedLists` `a` and `b` and returns a `LinkedList` with the elements of `b` after the elements of `a`.

```
pub fn append(a: LinkedList(t), b: LinkedList(t)) -> LinkedList(t) {
  case a {
    Empty -> ____________
    Link(first, rest) -> ________________________________________
  }
}
```

```
pub fn append(a: LinkedList(t), b: LinkedList(t)) -> LinkedList(t) {
  case a {
    Empty -> b
    Link(first, rest) -> Link(first, append(rest, b))
  }
}
```

This is an implementation of Gleam's `list.append` using our custom linked lists.

Note that we are creating a new `Link` for every element of `a`, but we are reusing the same `b` as the inputed list. In Python, this could be problematic because mutating `b` would also change every output of `append`, but because Gleam's data is immutable, there is no disadvantage to doing this and this is preferred for efficiency.

2. Implement `split`, which takes in a `LinkedList l` and a value `at`, and returns a tuple of the parts of `l` before and after the first occurrence of `at`. `at` is the first element of the second part. If `at` is not found, return `#(l, Empty)`.

```
pub fn split(l: LinkedList(a), at: a) -> #(LinkedList(a), LinkedList(a)) {
  case l {
    Empty -> #(Empty, Empty)
    Link(first, _) if first == at -> ________________________
    Link(first, rest) -> {
      let #(before, after) = ________________________
      ____________________________________
    }
  }
}
```

```
pub fn split(l: LinkedList(a), at: a) -> #(LinkedList(a), LinkedList(a)) {
  case l {
    Empty -> #(Empty, Empty)
    Link(first, _) if first == at -> #(Empty, l)
    Link(first, rest) -> {
      let #(before, after) = split(rest, at)
      #(Link(first, before), after)
    }
  }
}
```

Following a similar logic as `append`, we build onto the list at the first element of the tuple until `at` is found, at which point we stop recursing and put the rest of the list at the second element of the tuple. The case where `at` is not found occurs naturally when `split` is eventually called on an empty list, at which point we would have the entire list in the first element and we simply put `Empty` as the second element.

The use of **`if`** here is called a guard. Instead of splitting into cases like in Python, it is a further specification of a case, so the pattern `Link(first, _)` **`if`** `first == at` is matched only when `first == at`.

3. Implement `invert`, which takes in a `LinkedList l` and a value `at`, and moves the elements starting from the first occurrence of `at` to the front, before the elements that came before `at`. For example, inverting `1 -> 2 -> 3 -> 4` at `3` gives `3 -> 4 -> 1 -> 2`. If `at` is not found, return `l` unchanged. Use `split` and `append`.

```
pub fn invert(l: LinkedList(a), at: a) -> LinkedList(a) {
  let #(before, after) = ______________________
  ______________________
}
```

```
pub fn invert(l: LinkedList(a), at: a) -> LinkedList(a) {
  let #(before, after) = split(l, at)
  append(after, before)
}
```

We first `split` at `at`, then `append` `before` after `after`.

Just like in Python, a `Tree` in Gleam contains a `label` and a list of `Tree`s as `branches`. `label` can be of a generic type, called `a` in our implementation. `leaf` returns a `Tree` with label `label` and an empty list for `branches`.

```
pub type Tree(a) {
    Tree(label: a,
        branches: List(Tree(a)))
}

pub fn leaf(label: a) -> Tree(a) {
    Tree(label, [])
}
```

You can find a list of useful builtin functions in Gleam here:
https://gleam-stdlib.hexdocs.pm/gleam/list.html
https://gleam-stdlib.hexdocs.pm/gleam/int.html
https://gleam-stdlib.hexdocs.pm/gleam/result.html

4. Implement `prune_even`, which prunes every branch in a tree that has an even label. Odd child branches of branches with even labels are not preserved.

```
pub fn prune_even(tree: Tree(Int)) -> Tree(Int) {
    let Tree(label, branches) = tree

    let remove_even = list.filter(branches, _____________________________)
    let prune_children = list._________(_____________________________)

    Tree(label, prune_children)
}
```

```
pub fn prune_even(tree: Tree(Int)) -> Tree(Int) {
    let Tree(label, branches) = tree

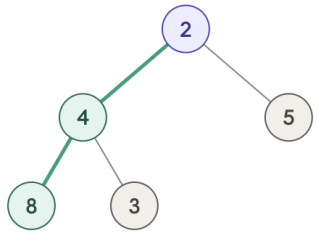
    let remove_even =
        list.filter(branches, fn(branch) { branch.label % 2 != 0 })
    let prune_children = list.map(remove_even, prune_even)
    Tree(label, prune_children)
}
```

We start with `let Tree(label, branches)= tree` so that we can assign the tree's label and branches to a name that we're able to reference in code. If we didn't do that, we wouldn't be able to explicitly reference the tree's label and branches.

We're going to do this in a similar way to a typical 61a solution, so basically filter out even branches and then recursively call the function on the remaining branches. Data is immutable in Gleam, so we can't mutate the tree, we have to make a new one. Because of this, we make a new list of all our branches after being pruned. We have to do **`list.filter`**... instead of just calling **`filter`**. This is the case with all the standard list operations (like **`max`** and **`sum`** too) in Gleam. The syntax for a lambda is `fn(args){ body }`, and it works just like a python lambda.

After this we're going to recursively call the function on the rest of the branches. We can't do this with a for loop because Gleam doesn't have iteration, just recursion. However, we can do **`list.map`**. The prune and recurse steps could be done in the same line with `let prune_children = list.map(list.filter(branches, fn(branch){ branch.label % 2 != 0 }), prune_even)`. Once we're done that, we just return our new tree. We don't need to put a return statement, it will return the last statement evaluated.

5. A *treemil* is a tree in which at least one of the child node(s) of each non-leaf node has a label that is a multiple of the label of the node. Alternatively, a treemil is a tree where every node has at least one path from the node to a leaf consisting entirely of nodes that are multiples of its parent. Implement `is_treemil`, which returns whether a tree is a treemil.




```
pub fn is_treemil(tree: Tree(Int)) -> Bool {
    // valid if:
    // 1. at least one child node is a multiple of its parent
    // 2. all children are valid treemils
    // leaves are valid treemils

    case tree {
        //is a leaf
        Tree(_, []) -> _________

        //is not a leaf
        Tree(label, branches) -> {
            let child_multiples = list._______(_______, ________________)
            case list._________(child_multiples) >= 1 {
                True -> list._________(________________, ________________)
                False -> _________
            }
        }
    }
}
```

This is a rewrite of the Tree recursion question treeil.

We use a `case` statement which is basically an if statement, but you have to cover every case of the object you're referencing. `Tree(_, [])` means a tree is a leaf: `_` means that we don't care what's in the first element, and `[]` means that we're looking for a Tree where the branches list is empty.

```
pub fn is_treemil(tree: Tree(Int)) -> Bool {
    case tree {
        Tree(_, []) -> True

        Tree(label, branches) -> {
            let child_multiples =
                list.filter(branches, fn(b) { b.label % label == 0 })
            case list.length(child_multiples) >= 1 {
                True -> list.all(branches, is_treemil)
                False -> False
            }
        }
    }
}
```

6. Implement `max_treemil` which takes in a tree `t` and determines the highest number of nodes in a treemil that is a subtree of `t`. You may assume `is_treemil` is implemented correctly from the previous question.

```
pub fn max_treemil(tree: Tree(Int)) -> Int {
    //Given a tree (that is not necessarily a treemil), returns
    //the highest number of nodes in a treemil in the tree.

    case tree {
        //is a leaf
        Tree(_, []) -> _________

        //is not a leaf
        Tree(_, branches) -> {
            case ___________________________________ {
                True -> 1 + int._________(___________________________________)
                False -> {
                    let sizes = list._________(_________________, _________________)
                    result.unwrap(list.max(_________, int.compare), 0)
                }
            }
        }
    }
}
```

```
pub fn max_treemil(tree: Tree(Int)) -> Int {
    case tree {
        Tree(_, []) -> 1

        Tree(_, branches) -> {
            case is_treemil(tree) {
                True -> 1 + int.sum(list.map(branches, max_treemil))
                False -> {
                    let sizes = list.map(branches, max_treemil)
                    result.unwrap(list.max(sizes, int.compare), 0)
                }
            }
        }
    }
}
```

We use **`int.sum`** instead of just **`sum`** the **`sum`** builtin function is found in the **`int`** module.

We had to do `result.unwrap` because the **`list.max`** function doesn't return a number, it returns a `Result`, which is an `Ok(value)` or `Error(reason)`. `Error(reason)` is returned in case the **`max`** function was called on an empty list. We know that we're not calling it on an empty list, but the function doesn't know that. `result.unwrap` says "I want to use the value from this **`list.max`** function if it was a valid result, and if the function errored we'll use the number in the second argument", which we put as 0. This means, we're using the max of the branches, and if the branches list was empty for some reason we'll use zero instead.